



DESENVOLVIMENTO FULL STACK

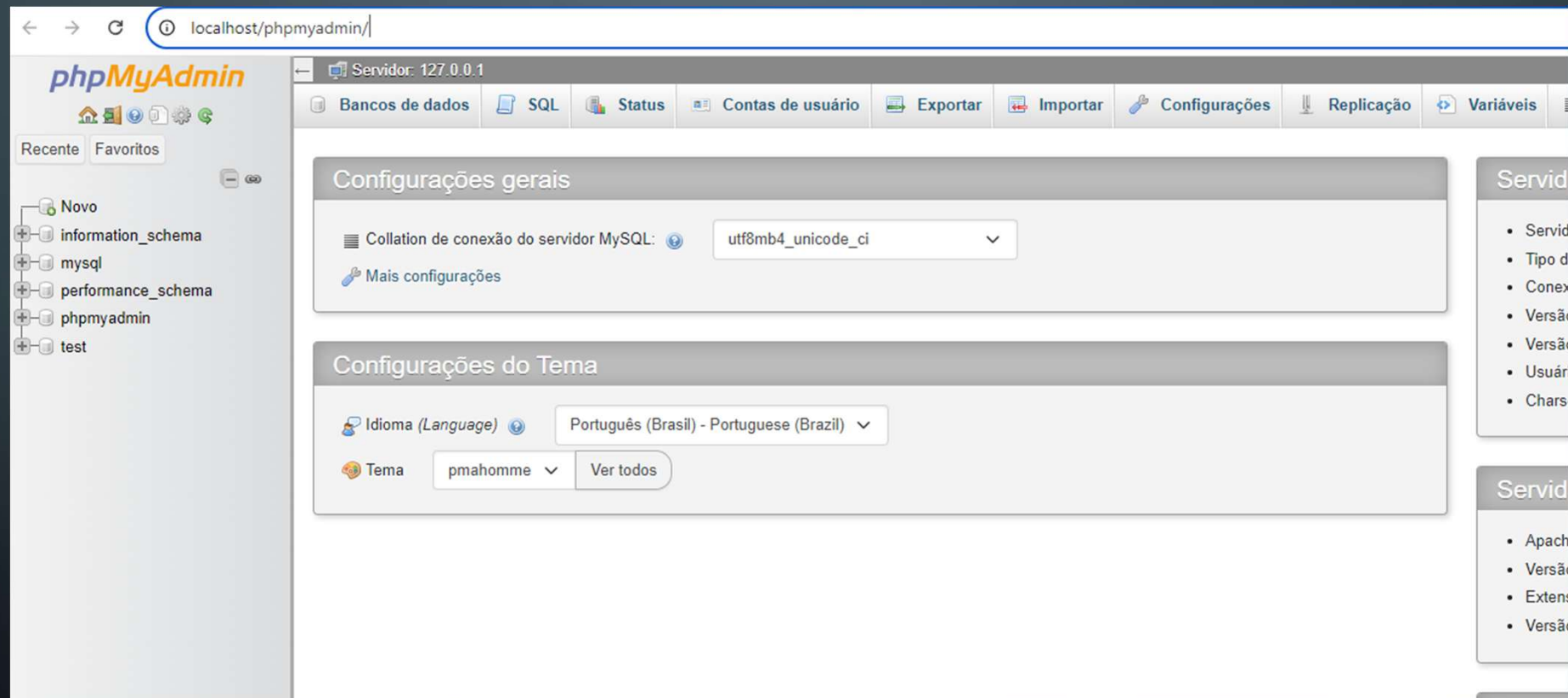
ALEX ZACHARIAS

ALEX@EINSTEINLIMEIRA.COM.BR

AMBIENTE PARA O BACK END

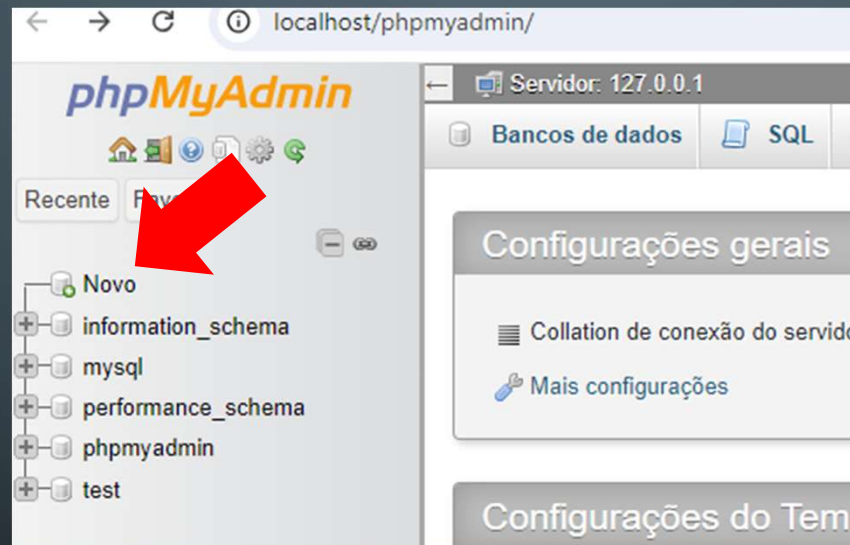
- Iremos utilizar o PHP com MySQL no Back End, para isto instale o XAMPP no seu computador.
 - Acesse em https://www.apachefriends.org/pt_br/index.html
- Primeiro vamos utilizar o PHPMyAdmin para criar nosso banco de dados:
 - Acesse <http://localhost/phpmyadmin/>

MONTANDO O BANCO DE DADOS



MONTANDO O BANCO DE DADOS

- Clique em Novo para criar o banco de dados:



MONTANDO O BANCO DE DADOS

- Coloque um nome no seu banco de dados (utilizei db_primeiro_banco), e defina a coleção (collate) **utf8_general_ci**



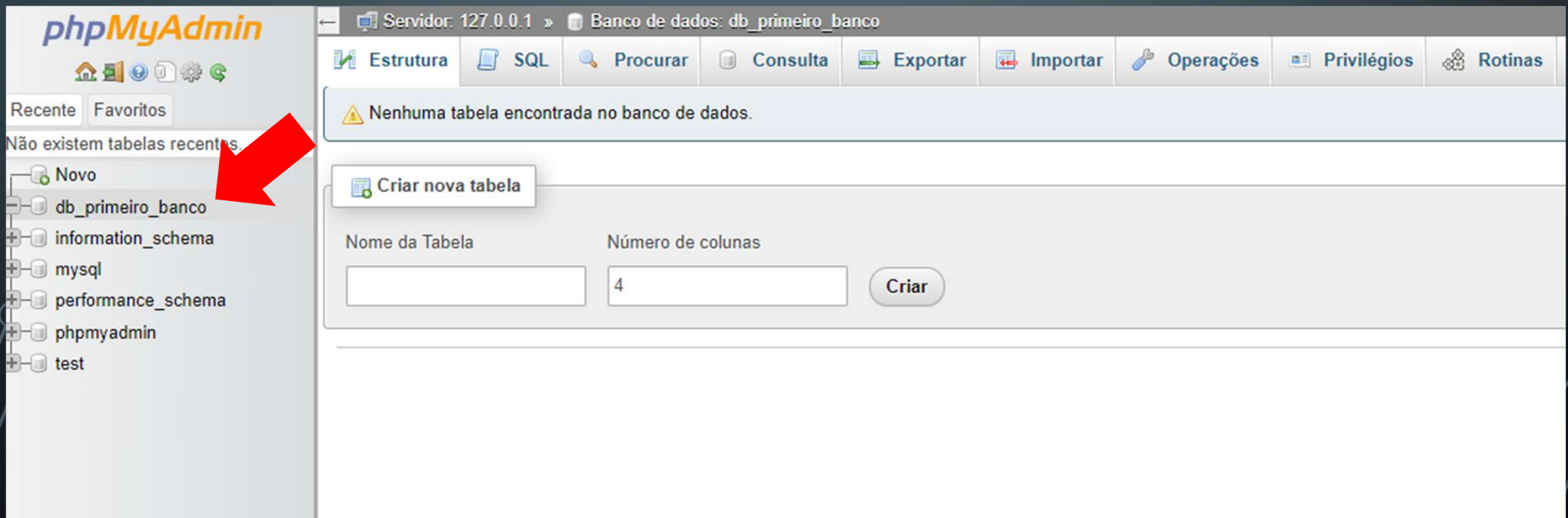
- Clique em **Criar**

MONTANDO O BANCO DE DADOS

- A Coleção define a acentuação e os tipos de caracteres utilizados.
- O `utf8_general_ci` é o ideal para os caracteres da língua portuguesa.
- Utilizar outra coleção pode gerar problemas ao utilizar caracteres com acentuação.

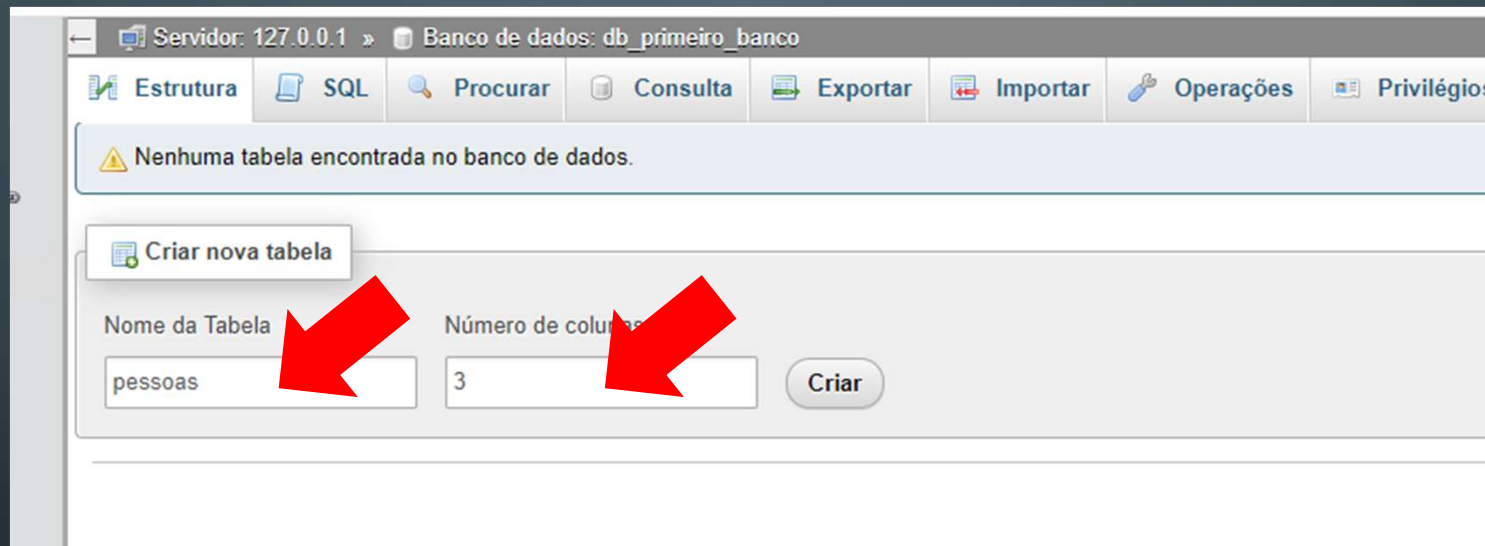
MONTANDO O BANCO DE DADOS

- Após a criação o banco irá aparecer na lista do lado esquerdo da tela:



MONTANDO O BANCO DE DADOS

- Vamos criar nossa tabela:



The screenshot shows a web-based database management interface. At the top, the browser address bar displays 'Servidor: 127.0.0.1' and 'Banco de dados: db_primeiro_banco'. Below the address bar is a navigation menu with tabs: 'Estrutura', 'SQL', 'Procurar', 'Consulta', 'Exportar', 'Importar', 'Operações', and 'Privilégios'. A message box below the menu states: 'Nenhuma tabela encontrada no banco de dados.' Below this is a section titled 'Criar nova tabela'. It contains two input fields: 'Nome da Tabela' with the value 'pessoas' and 'Número de colunas' with the value '3'. Two large red arrows point to these input fields. To the right of the input fields is a button labeled 'Criar'.

- Informe o nome da tabela, quantidade de colunas e clique em **Criar**.

MONTANDO O BANCO DE DADOS

- Defina os campos, tipos e tamanhos quando necessário, marque o ID como chave primaria, ao final clique em **Salvar**:

The screenshot shows a database creation interface. At the top, there's a form with 'Nome da Tabela: pessoas', 'Adicionar 1 coluna(s)', and an 'Executar' button. Below this is a table with columns: Nome, Tipo, Tamanho/Valores, Padrão, Colaço, Atributos, Nulo, Índice, and A.I. The table contains three rows: 'id' (INT, Nenhum, Nenhum, PRIMARY), 'nome' (VARCHAR, 80, Nenhum, Nenhum), and 'idade' (INT, Nenhum, Nenhum, Nenhum). A red box highlights the first three rows. A red arrow points to the 'PRIMARY' label in the 'Índice' column for the 'id' row. Below the table, there's a 'Mecanismo de armazenamento:' section with a dropdown set to 'InnoDB'. At the bottom, there's a 'Definição da PARTIÇÃO:' section with a dropdown for 'Partição por:' and a text input for '(Expressão ou lista de coluna)'. Below that is a 'Partições:' text input. At the very bottom, there are two buttons: 'Ver SQL' and 'Salvar'. A red arrow points to the 'Salvar' button.

Nome	Tipo	Tamanho/Valores	Padrão	Colaço	Atributos	Nulo	Índice	A.I
id	INT		Nenhum			☐	PRIMARY	☐
nome	VARCHAR	80	Nenhum			☐	---	☐
idade	INT		Nenhum			☐	---	☐

Mecanismo de armazenamento: InnoDB

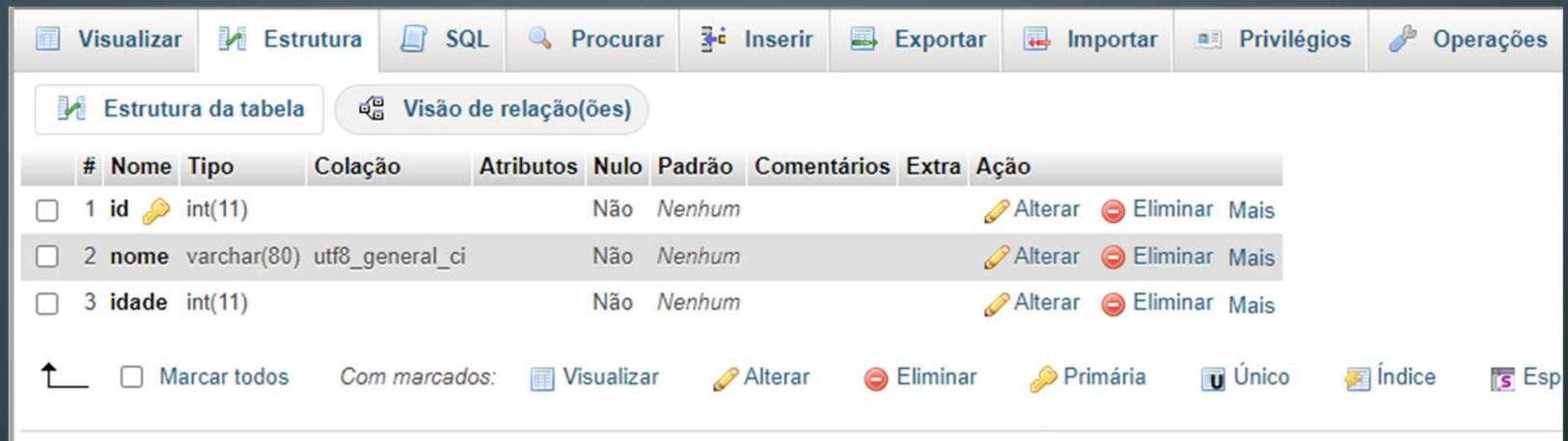
Definição da PARTIÇÃO:

Partição por: (Expressão ou lista de coluna)

Partições:

Ver SQL Salvar

MONTANDO O BANCO DE DADOS



The screenshot shows a database management interface with a top menu bar containing: Visualizar, Estrutura, SQL, Procurar, Inserir, Exportar, Importar, Privilégios, and Operações. Below the menu, there are two tabs: 'Estrutura da tabela' (selected) and 'Visão de relação(ões)'. The main area displays a table structure with the following columns: #, Nome, Tipo, Colaço, Atributos, Nulo, Padrão, Comentários, Extra, and Ação. The table contains three rows of data:

#	Nome	Tipo	Colaço	Atributos	Nulo	Padrão	Comentários	Extra	Ação
1	id	int(11)			Não	Nenhum			Alterar Eliminar Mais
2	nome	varchar(80)	utf8_general_ci		Não	Nenhum			Alterar Eliminar Mais
3	idade	int(11)			Não	Nenhum			Alterar Eliminar Mais

At the bottom of the interface, there is a toolbar with the following options: Marcar todos, Com marcados: Visualizar, Alterar, Eliminar, Primária, Único, Índice, and Esp.

- Note que a coleção foi definida automaticamente apenas para o campo nome, pois esse é o único campo que contém caracteres que podem ter acentuação.

MONTANDO O BANCO DE DADOS

- Podemos incluir alguns dados clicando em SQL e digitando o comando, ao final clique em **Executar**:

The screenshot shows a database management tool interface. At the top, there is a menu bar with options: Visualizar, Estrutura, SQL, Procurar, Inserir, Exportar, Importar, Privilégios, Operações, and Acionadores. A red arrow points to the 'SQL' option. Below the menu bar, there is a text box that says 'Executar consulta(s) SQL na tabela db_primeiro_banco.pessoas:'. Below this, there is a code editor with three lines of SQL code, which are highlighted with a red box:

```
1 INSERT INTO pessoas(id, nome, idade) VALUES (1,'Huguinho',18);  
2 INSERT INTO pessoas(id, nome, idade) VALUES (2,'Zezinho',17);  
3 INSERT INTO pessoas(id, nome, idade) VALUES (3,'Luizinho',19);
```

Below the code editor, there are buttons for 'SELECT *', 'SELECT', 'INSERT', 'UPDATE', 'DELETE', 'Limpar', 'Formato', and 'Obter consulta auto-salva'. There is also a checkbox for 'Unir parâmetros'. At the bottom, there is a row of options: 'Delimitadores' with a semicolon symbol, 'Mostrar esta consulta SQL aqui novamente', 'Manter caixa de consulta', 'Recuar quando finalizado', 'Habilitar verificação de chaves estrangeiras' (checked), and a large 'Executar' button. A red arrow points to the 'Executar' button.

MONTANDO O BANCO DE DADOS

- Clique em visualizar para ver o conteúdo da tabela:

The screenshot shows a database management interface with a top navigation bar containing icons and labels for 'Visualizar', 'Estrutura', 'SQL', 'Procurar', 'Inserir', 'Exportar', 'Importar', 'Privilégios', 'Operações', and 'Ações'. Below the navigation bar, a green status bar indicates 'Mostrando registros 0 - 2 (3 no total, Consulta levou 0.0003 segundos.)'. The main content area displays the SQL query 'SELECT * FROM `pessoas`' and a row of action links: 'Perfil', 'Editar em linha', 'Editar', 'Demonstrar SQL', 'Criar código PHP', and 'Atualizar'. Below these links is a control bar with a 'Mostrar tudo' checkbox, a 'Número de linhas' dropdown set to '25', a 'Filtrar linhas' input field with the placeholder 'Procurar nesta tabela', and an 'Ordenar pela chave' dropdown set to 'Nenhum'. An 'Opções extras' button is located below the control bar. The table data is displayed with columns 'id', 'nome', and 'idade'. Each row has a checkbox, an 'Editar' button, a 'Copiar' button, and a 'Remover' button. The data rows are: (1, Huguinho, 18), (2, Zezinho, 17), and (3, Luizinho, 19). At the bottom, there is another control bar identical to the one above, including a 'Marcar todos' checkbox and a 'Com marcados:' section with 'Editar', 'Copiar', 'Remover', and 'Exportar' buttons.

Visualizar Estrutura SQL Procurar Inserir Exportar Importar Privilégios Operações Ações

✓ Mostrando registros 0 - 2 (3 no total, Consulta levou 0.0003 segundos.)

SELECT * FROM `pessoas`

☐ Perfil [[Editar em linha](#)] [[Editar](#)] [[Demonstrar SQL](#)] [[Criar código PHP](#)] [[Atualizar](#)]

☐ Mostrar tudo | Número de linhas: 25 | Filtrar linhas: Procurar nesta tabela | Ordenar pela chave: Nenhum

Opções extras

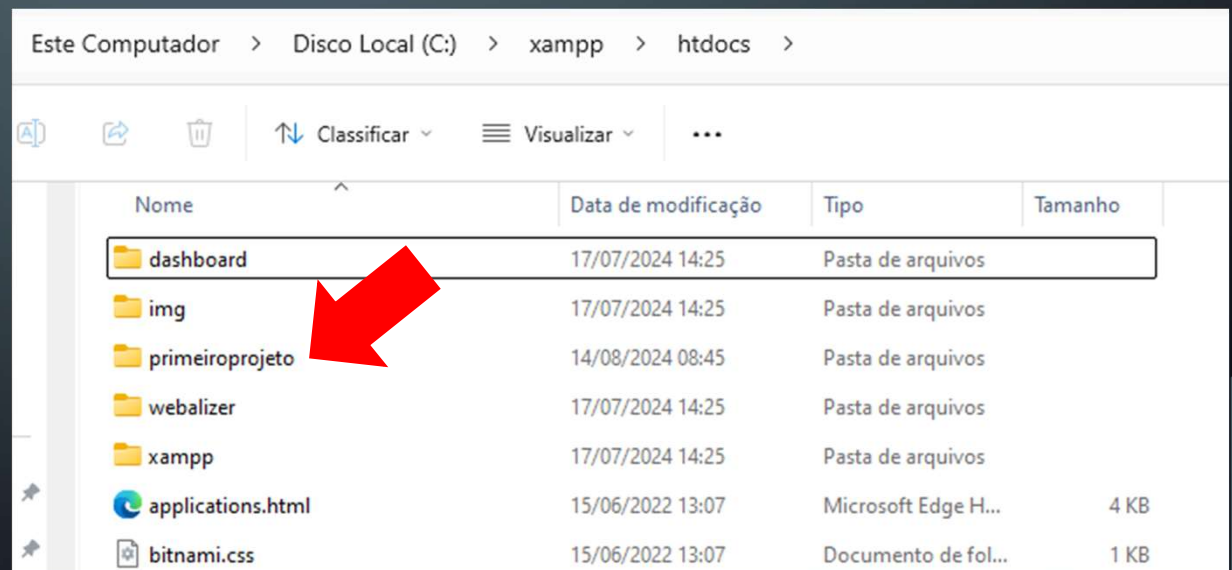
				id	nome	idade
☐	Editar	Copiar	Remover	1	Huguinho	18
☐	Editar	Copiar	Remover	2	Zezinho	17
☐	Editar	Copiar	Remover	3	Luizinho	19

↑ ☐ Marcar todos | Com marcados: [Editar](#) [Copiar](#) [Remover](#) [Exportar](#)

☐ Mostrar tudo | Número de linhas: 25 | Filtrar linhas: Procurar nesta tabela | Ordenar pela chave: Nenhum

BACK END - PHP

- Como estamos utilizando o XAMPP, a pasta padrão para colocarmos nossos projetos é **C:\xampp\htdocs**
- Vamos criar uma pasta dentro dela, vou chamar de **primeiroprojeto**:



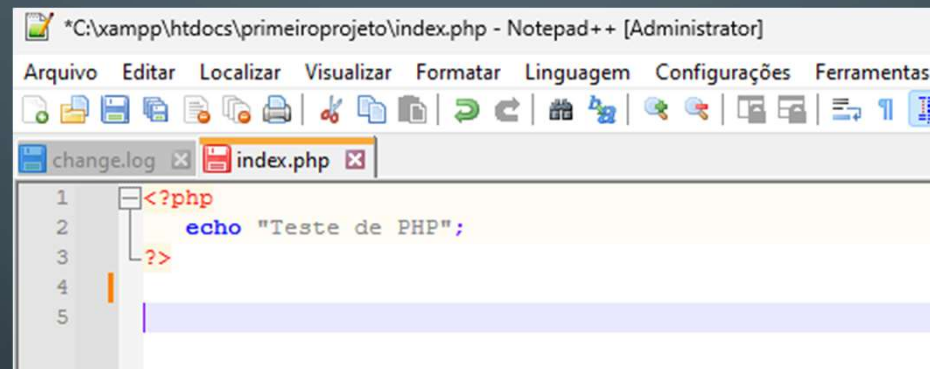
BACK END - PHP

- Para acessar nossa pasta basta digitar no navegador **localhost/primeiroprojeto**, como não tem nada na pasta, ele exibe a tela abaixo.



BACK END - PHP

- Dentro da pasta criada, crie um arquivo de nome index.php com o seguinte conteúdo:

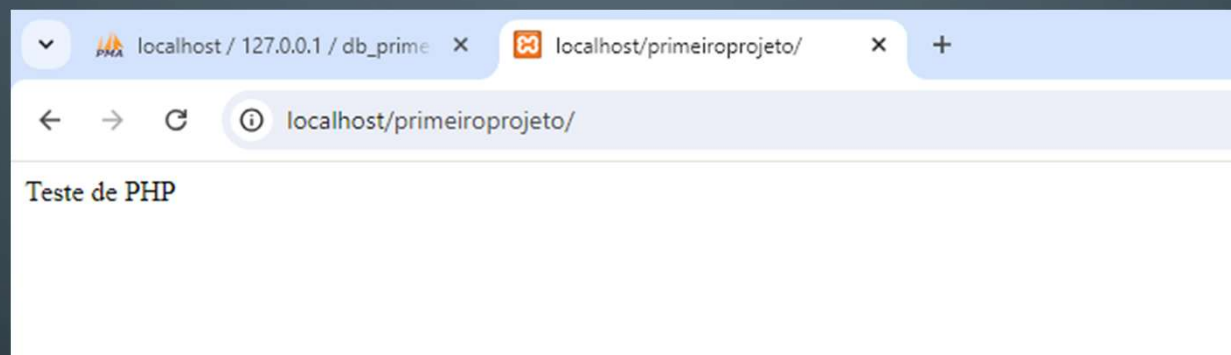
A screenshot of a Notepad++ window titled '*C:\xampp\htdocs\primeiroprojeto\index.php - Notepad++ [Administrator]'. The window has a menu bar with 'Arquivo', 'Editar', 'Localizar', 'Visualizar', 'Formatar', 'Linguagem', 'Configurações', and 'Ferramentas'. Below the menu is a toolbar with various icons. The tab bar shows 'change.log' and 'index.php'. The main text area contains the following PHP code:

```
1 <?php
2     echo "Teste de PHP";
3 ?>
```

- O nome do arquivo deve ser esse para o servidor web abri-lo automaticamente ao passarmos apenas a pasta para o navegador.

BACK END - PHP

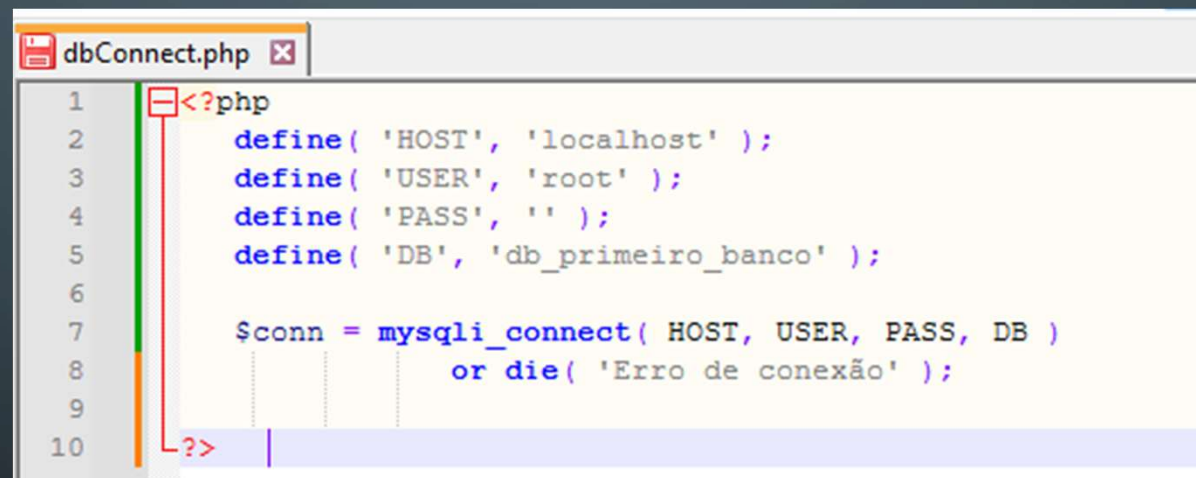
- Acesse novamente aquele endereço no navegador:



- Nosso programa funcionou.

BACK END – PHP – ACESSO AO BANCO DE DADOS

- Vamos criar um novo arquivo php para fazer o acesso ao banco de dados:

A screenshot of a code editor window titled 'dbConnect.php'. The code is written in PHP and defines database connection constants and a connection function. The code is as follows:

```
1 <?php
2     define( 'HOST', 'localhost' );
3     define( 'USER', 'root' );
4     define( 'PASS', '' );
5     define( 'DB', 'db_primeiro_banco' );
6
7     $conn = mysqli_connect( HOST, USER, PASS, DB )
8         or die( 'Erro de conexão' );
9
10 ?>
```

- Chamei o arquivo de **dbConnect.php**

BACK END – PHP – ACESSO AO BANCO DE DADOS

- Entendendo o código:

```
define( 'HOST', 'localhost' );
```

- A função **define** cria uma variável passando um valor para a mesma.

```
$conn = mysqli_connect( HOST, USER, PASS, DB )  
or die( 'Erro de conexão' );
```

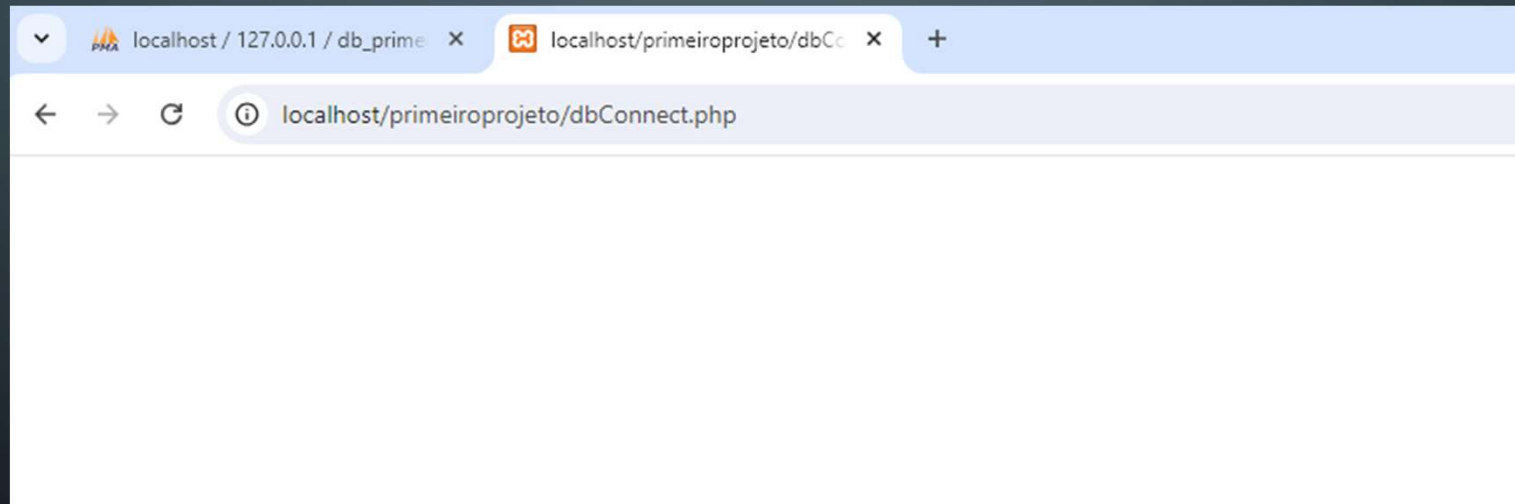
- **mysqli_connect** faz a conexão ao banco de dados, a função **die** exibe uma mensagem se der algum erro.

BACK END – PHP – ACESSO AO BANCO DE DADOS

- Para se conectar ao banco de dados precisamos de quatro campos:
 - **HostName:** nome ou IP do computador onde esta o banco de dados;
 - **User:** usuário do banco de dados;
 - **Password:** senha do usuário do banco de dados;
 - **DataBase:** nome do banco de dados.

BACK END – PHP – ACESSO AO BANCO DE DADOS

- Execute o arquivo criado, se não tiver nenhum erro, ele vai exibir uma página em branco:



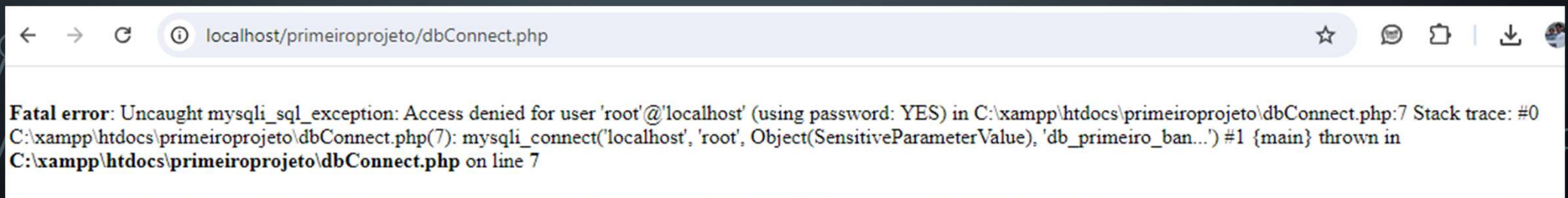
BACK END – PHP – ACESSO AO BANCO DE DADOS

- Vamos simular um erro, vou colocar uma senha errada:

```
<?php
define( 'HOST', 'localhost' );
define( 'USER', 'root' );
define( 'PASS', 'sdfadf' );
define( 'DB', 'db_primeiro_banco' );

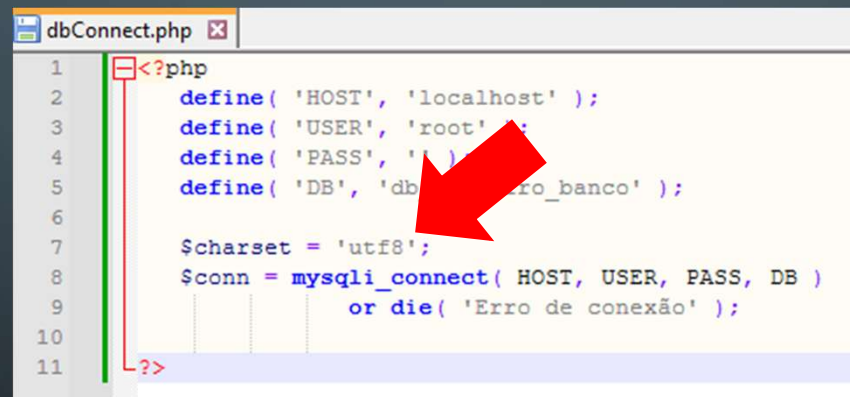
$conn = mysqli_connect( HOST, USER, PASS, DB )
or die( 'Erro de conexão' );
```

- Erro da senha:



BACK END – PHP – ACESSO AO BANCO DE DADOS

- Vamos incluir uma variável para definir o charset (coleção) dentro do nosso arquivo de conexão ao banco:



```
dbConnect.php
1  <?php
2      define( 'HOST', 'localhost' );
3      define( 'USER', 'root' );
4      define( 'PASS', '' );
5      define( 'DB', 'db_novo_banco' );
6
7      $charset = 'utf8';
8      $conn = mysqli_connect( HOST, USER, PASS, DB )
9              or die( 'Erro de conexão' );
10
11  ?>
```

A red arrow points to the line `$charset = 'utf8';` in the code snippet.

BACK END – CRIANDO A PRIMEIRA API

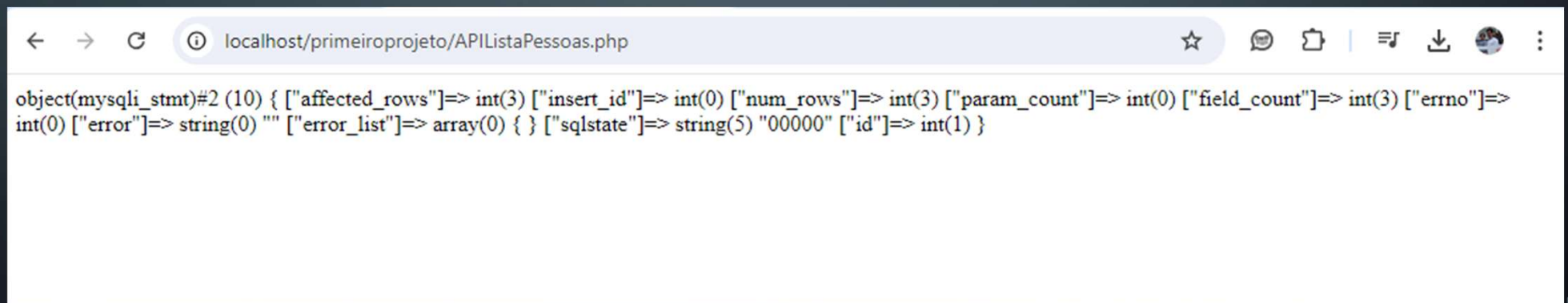
- Agora crie um novo arquivo de nome **APIListaPessoas.php** que irá fornecer os registros da tabela para o Front End (o código esta no próximo slide).
- O comando **require_once('dbConnect.php')** executa o arquivo de conexão ao banco e trás as variáveis criadas naquele arquivo para o arquivo atual.
- O comando **var_dump** só foi utilizado para teste do código.

BACK END – CRIANDO A PRIMEIRA API

```
APIListaPessoas.php
1  <?php
2      //Conexão ao banco
3      require_once('dbConnect.php');
4
5      //Define a coleção na nossa Conexão
6      mysqli_set_charset($conn, $charset);
7
8      $query = 'SELECT id, nome, idade FROM pessoas';
9
10     //Prepara a consulta ao banco de dados
11     $stmt = mysqli_prepare($conn, $query);
12
13     //Executa e armazena a query na memoria
14     mysqli_stmt_execute($stmt);
15     mysqli_stmt_store_result($stmt);
16
17     //associa os campos da query a variaveis aqui no php
18     mysqli_stmt_bind_result($stmt, $id, $nome, $idade);
19
20     //apenas para teste de funcionamento, vamos dar um dump na variavel
21     var_dump($stmt);
22
23  ?>
```


BACK END – CRIANDO A PRIMEIRA API

- Abra no navegador o programa criado, será exibido os dados da variável, com a quantidade de registros existentes:

A screenshot of a web browser window. The address bar shows the URL 'localhost/primeiroprojeto/APIListaPessoas.php'. The page content displays a PHP var_dump output for a MySQL statement object. The output shows details like 'affected_rows', 'insert_id', 'num_rows', 'param_count', 'field_count', 'errno', 'error', 'error_list', 'sqlstate', and 'id'.

```
object(mysqli_stmt)#2 (10) { ["affected_rows"]=> int(3) ["insert_id"]=> int(0) ["num_rows"]=> int(3) ["param_count"]=> int(0) ["field_count"]=> int(3) ["errno"]=> int(0) ["error"]=> string(0) "" ["error_list"]=> array(0) { } ["sqlstate"]=> string(5) "00000" ["id"]=> int(1) }
```

BACK END – CRIANDO A PRIMEIRA API

- Vamos enviar os dados lidos para um array e gerar um JSON com esse array:

```
17 //associa os campos da query a variaveis aqui no php
18 mysqli_stmt_bind_result($stmt, $id, $nome, $idade);
19
20 $response = array(); //criando um array para carregar os dados da tabela
21
22 //Carrega os dados em um array
23 if ( mysqli_stmt_num_rows($stmt) > 0 ) {
24     while ( mysqli_stmt_fetch($stmt) ) {
25         array_push($response, array( "id" => $id,
26                                     "nome" => $nome,
27                                     "idade" => $idade ));
28     }
29 }
30
31 //Envia para a tela o array em formato json
32 echo json_encode($response);
33
34 ?>
```

BACK END – CRIANDO A PRIMEIRA API

```
if ( mysqli_stmt_num_rows($stmt) > 0 ) {
```

- Checa se existem registros na tabela

```
while ( mysqli_stmt_fetch($stmt)) {
```

- Pega cada um dos registros e coloca nas variáveis associadas no comando `mysqli_stmt_bind_result`

```
array_push($response, array( "id" => $id,  
                             "nome" => $nome,  
                             "idade" => $idade ));
```

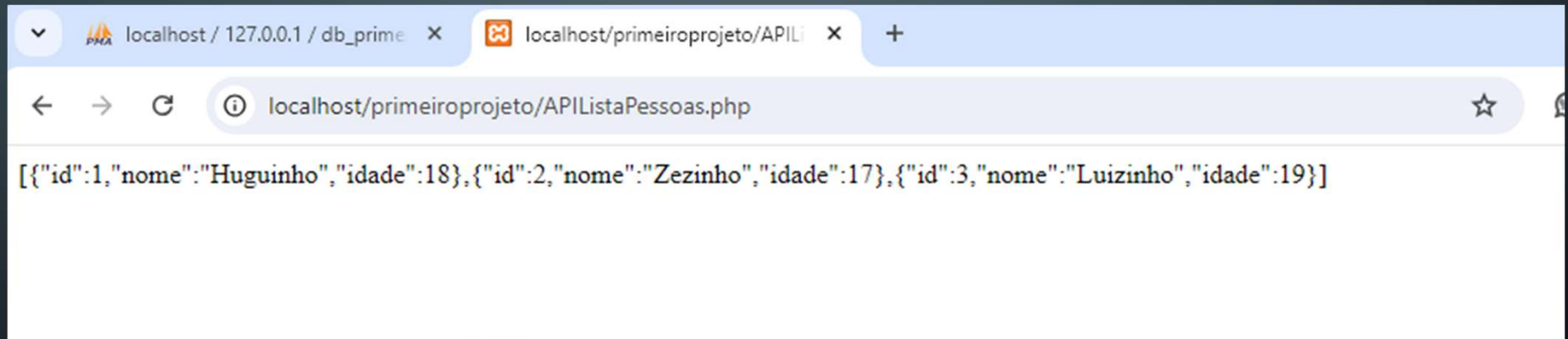
- Coloca o conteúdo dos campos no array de nome **\$response**

```
echo json_encode($response);
```

- Transforma o array em JSON e envia para o navegador.

BACK END – CRIANDO A PRIMEIRA API

- Agora nossa pagina retorna o arquivo JSON com os dados da tabela:



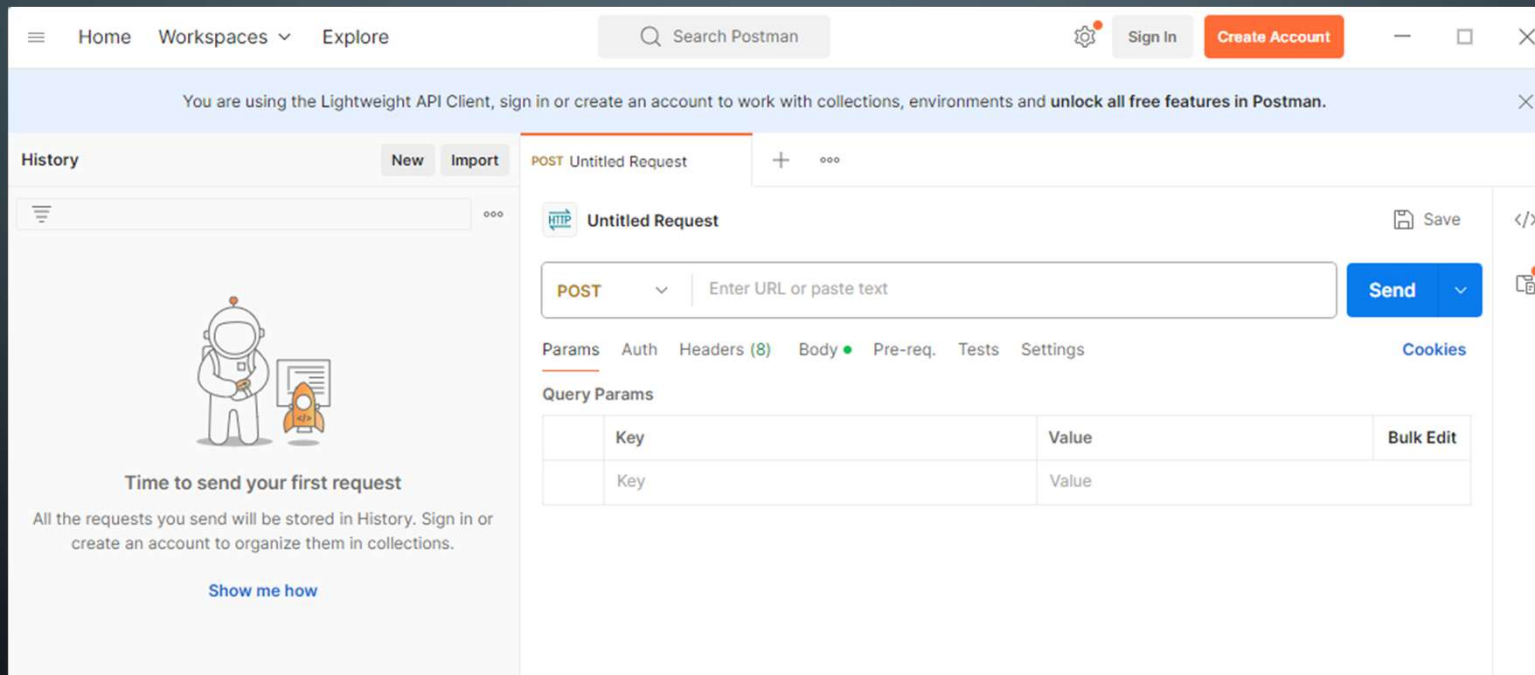
BACK END – POSTMAN



- O Postman é uma ferramenta para acesso e testes de APIs
- A API que criamos esta bem simples e sem recursos de segurança dos dados, sendo possível acessá-la diretamente do navegador pela URL dela.
- Ao criar uma API com as camadas de segurança ela não será acessível pelo navegador, aí entra o Postman, pois ele permite esse acesso para testes, inclusive podendo passar por ele senhas para acesso as API.

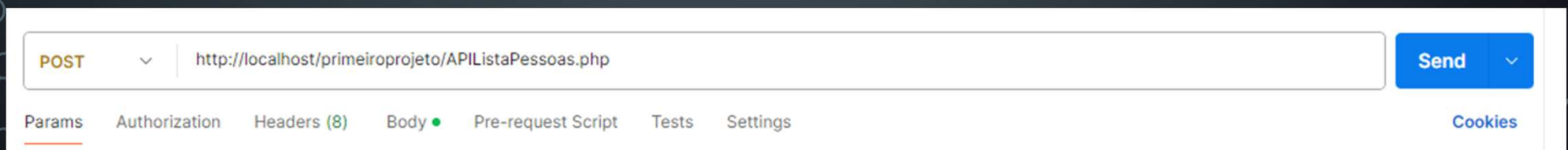
BACK END – POSTMAN

- Primeiramente vamos acessar nossa API pelo Postman.
- Abra o Postman:



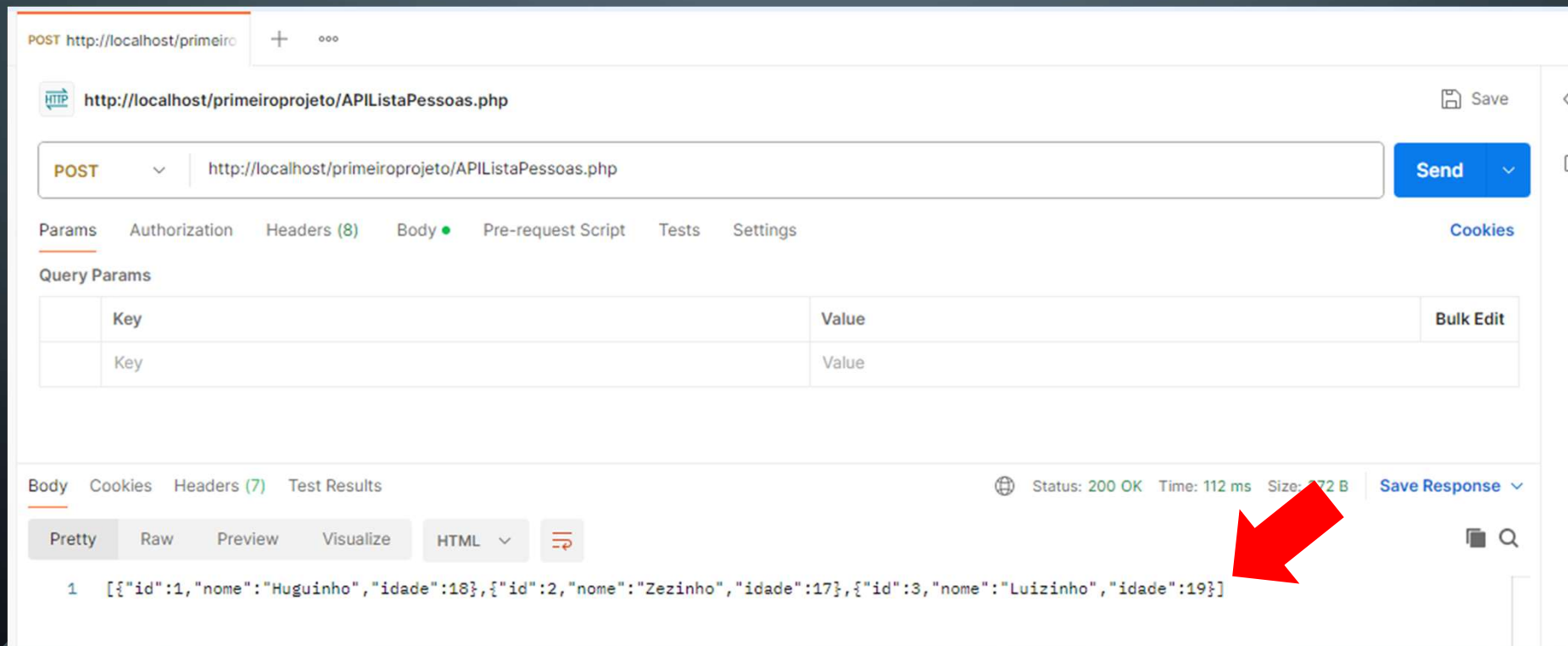
BACK END – POSTMAN

- Ele permite criar uma conta, onde ele ira armazenar as APIs que você esta desenvolvendo/testando, mas da para trabalhar com ele sem criar esse conta.
- Para acessar nossa API coloque o endereço na caixa de procura e clique em **Send**:



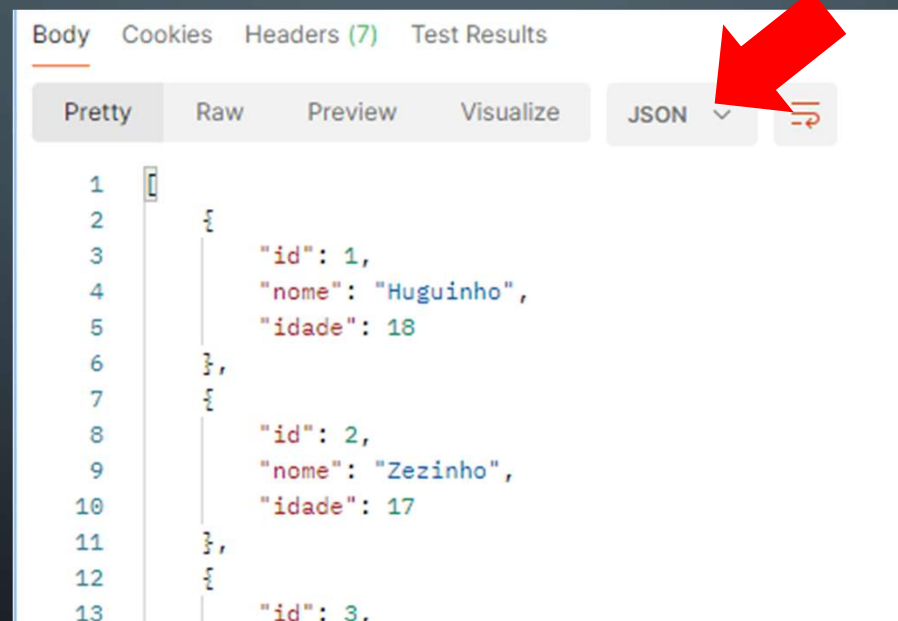
BACK END – POSTMAN

- Ele irá exibir os dados que a API retorna:



BACK END – POSTMAN

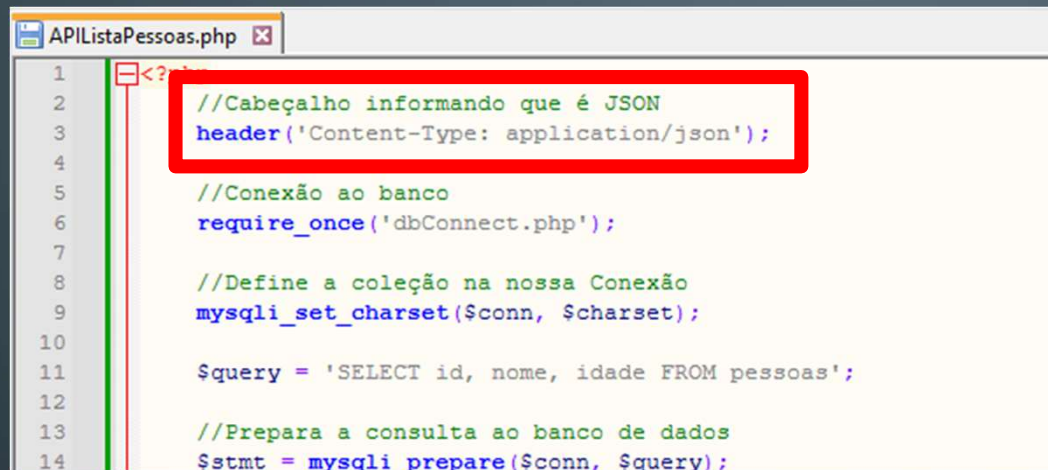
- Uma das facilidades é que ele permite mudar a saída para o padrão que estamos utilizando, no caso JSON:



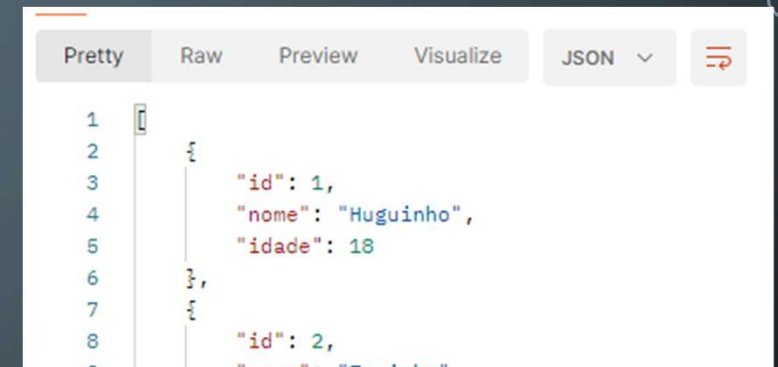
BACK END – HEADER

- Apesar de estar retornando um JSON, nossa API não está informando isso para quem vai receber, fazendo com os dados cheguem como se fosse um HTML, vamos muda-la para o receptor saiba que esse conteúdo é um JSON.
- Para isso vamos enviar um header (cabeçalho) no arquivo informando que se trata de um JSON.
- Altere o arquivo .php:

BACK END – HEADER

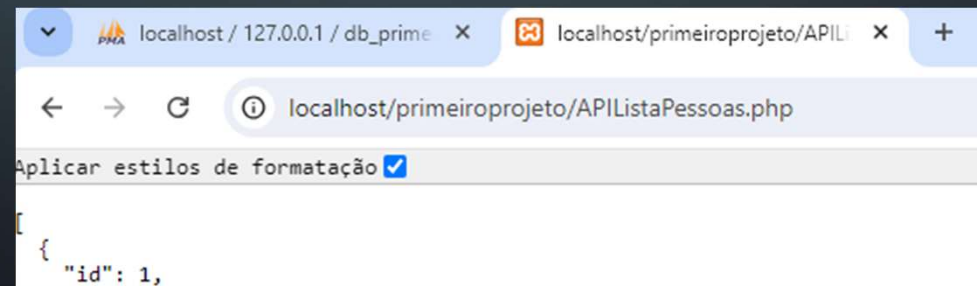


```
1 <?php
2 //Cabeçalho informando que é JSON
3 header('Content-Type: application/json');
4
5 //Conexão ao banco
6 require_once('dbConnect.php');
7
8 //Define a coleção na nossa Conexão
9 mysqli_set_charset($conn, $charset);
10
11 $query = 'SELECT id, nome, idade FROM pessoas';
12
13 //Prepara a consulta ao banco de dados
14 $stmt = mysqli_prepare($conn, $query);
```



```
1 [
2   {
3     "id": 1,
4     "nome": "Huguinho",
5     "idade": 18
6   },
7   {
8     "id": 2,
9     "nome": "Zezinho"
```

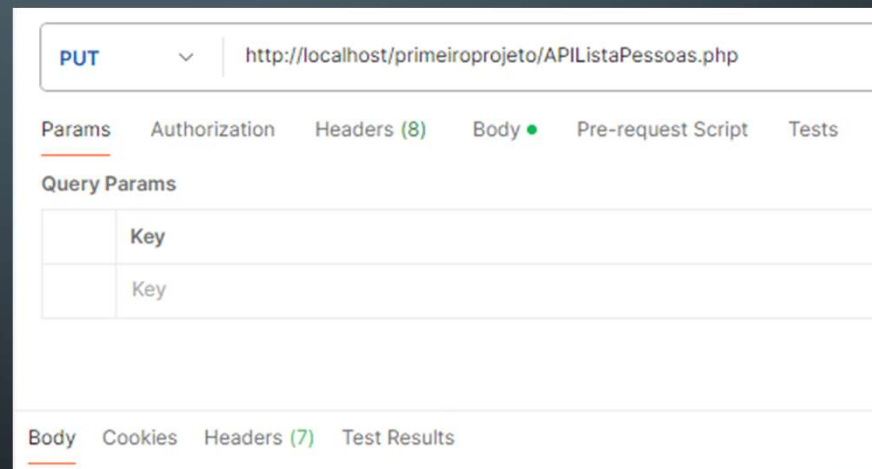
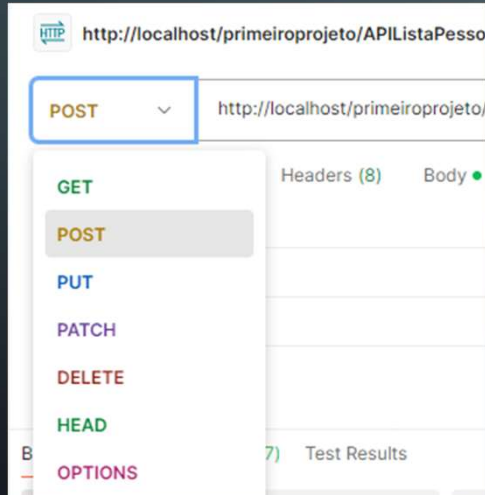
- Note que agora o Postman abre automaticamente no formato JSON, e o Chrome mostra uma opção para marcar o estilo de exibição:



```
localhost / 127.0.0.1 / db_prime x localhost/primeiroprojeto/API x +
localhost/primeiroprojeto/APIListaPessoas.php
Aplicar estilos de formatação [checked]
{
  "id": 1,
```

BACK END – POSTMAN

- Como não definimos o tipo de conexão que nossa API utiliza (aquelas conexões utilizadas em paginas web), ela esta respondendo em qualquer uma delas:



BACK END – REQUEST_METHOD

- Vamos alterar nossa API para que ela responda apenas um tipo de requisição, no caso as requisições **POST**.
- O PHP tem uma variável de nome **\$_SERVER** que retorna alguns dados da conexão feita, inclusive o tipo, para saber devemos testar o **REQUEST_METHOD** da variável **\$_SERVER**:

BACK END – REQUEST_METHOD

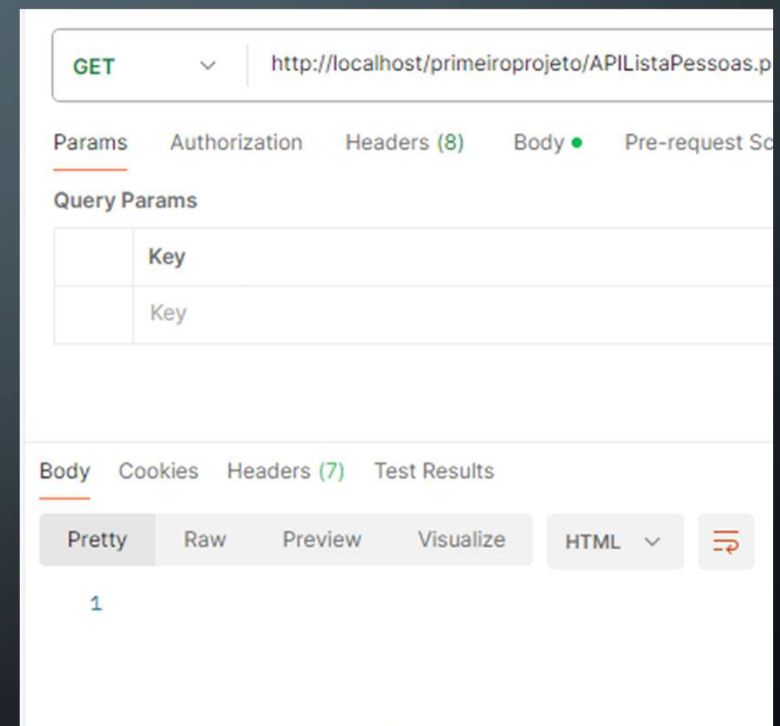
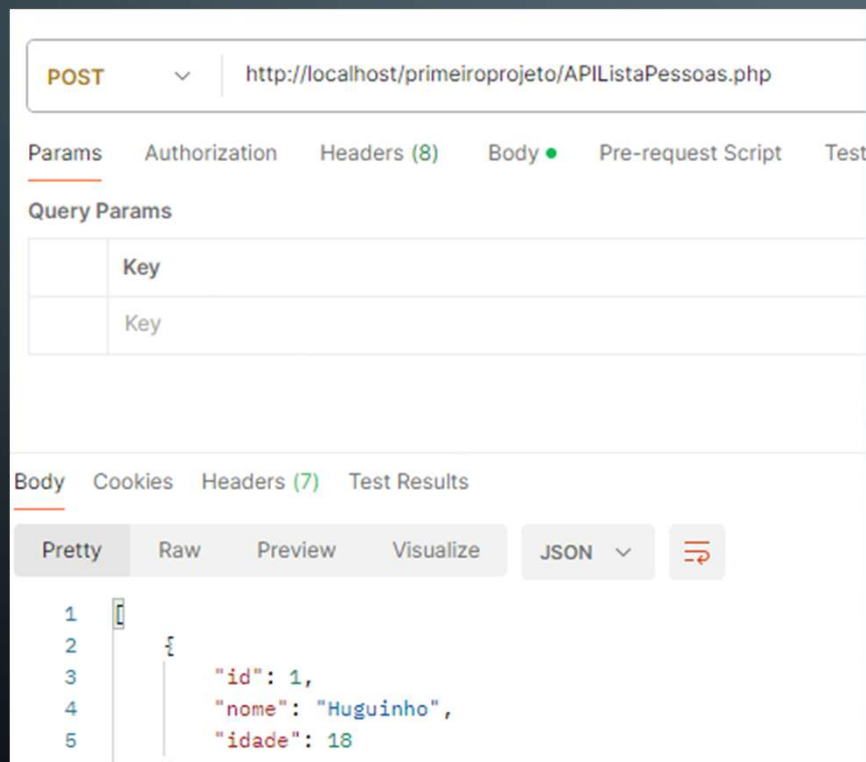
- Altere o código da API:

```
1 <?php
2 //Cabeçalho informando que é JSON
3 header('Content-Type: application/json');
4
5 if ( $_SERVER['REQUEST_METHOD'] == 'POST' )
6 {
7
8     require_once('dbConnect.php');
9
10    //Define a coleção na nossa Conexão
11    mysqli_set_charset($conn, $charset);
12
```

```
35
36 //Envia para a tela o array em formato json
37 echo json_encode($response);
38 }
39 ?>
```

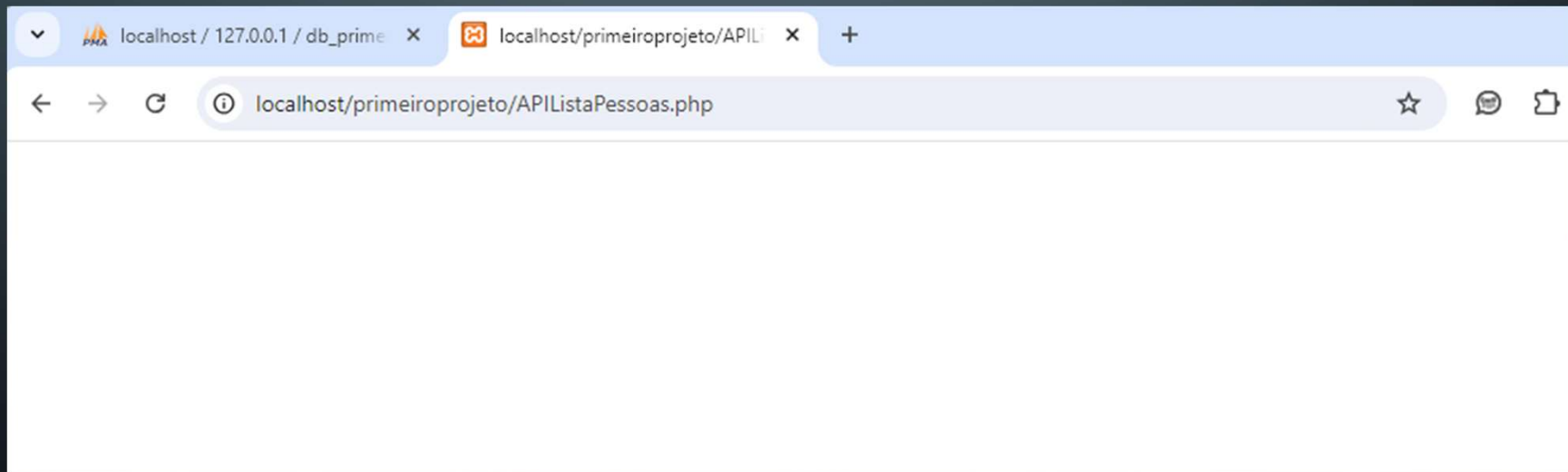
BACK END – REQUEST_METHOD

- Agora nossa API responde apenas para o método POST, qualquer outro tipo de solicitação recebe um arquivo em branco:



BACK END – REQUEST_METHOD

- Como o navegador utiliza o método GET por padrão, agora nossa API não responde mais quando for acessada por ele:



BACK END – REQUEST_METHOD

- Porque escolhemos o método POST?
- Ele permite que seja passado parâmetros na solicitação
- Nosso exemplo esta trazendo todos os dados da tabela, mas se quisermos trazer apenas um registro passando seu ID por exemplo, teremos que utilizar o método POST.

BACK END – TOKEN

- Já restringimos um pouco o acesso a nossa API, mas qualquer um que acessa a URL com uma solicitação POST consegue receber os dados.
- Existem vários métodos de autenticação para enviar usuário e senha ou token por exemplo.
- Vamos começar pelo mais simples de implementar, iremos enviar um token para validar o acesso, só recebe os dados quem tem o token.

BACK END – TOKEN

- Altere o código PHP:

```
1 <?php
2 //Cabeçalho informando que é JSON
3 header('Content-Type: application/json');
4
5 if ($_SERVER['REQUEST_METHOD'] == 'POST' )
6 {
7     $api_token = $_POST['api_token'];
8     if ($api_token == 'TokendeTeste')
9     {
10
11         //Conexão ao banco
12         require_once('dbConnect.php');
13
14         //Busca o token no banco de dados
15         $token = $db->query("SELECT token FROM usuarios WHERE token = '$api_token'");
16         $token = $token->fetch_assoc();
17
18         //Verifica se o token existe no banco de dados
19         if ($token) {
20             //Se existir, retorna o token
21             $response['token'] = $token['token'];
22             echo json_encode($response);
23         } else {
24             //Se não existir, retorna false
25             $response['auth_token'] = false;
26             echo json_encode($response);
27         }
28     }
29 }
30
31 //Envia para a tela o array em formato json
32 echo json_encode($response);
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49 ?>
```

BACK END – TOKEN

- Se voce tentar acessar nossa API pelo Postman, irá retornar um erro:

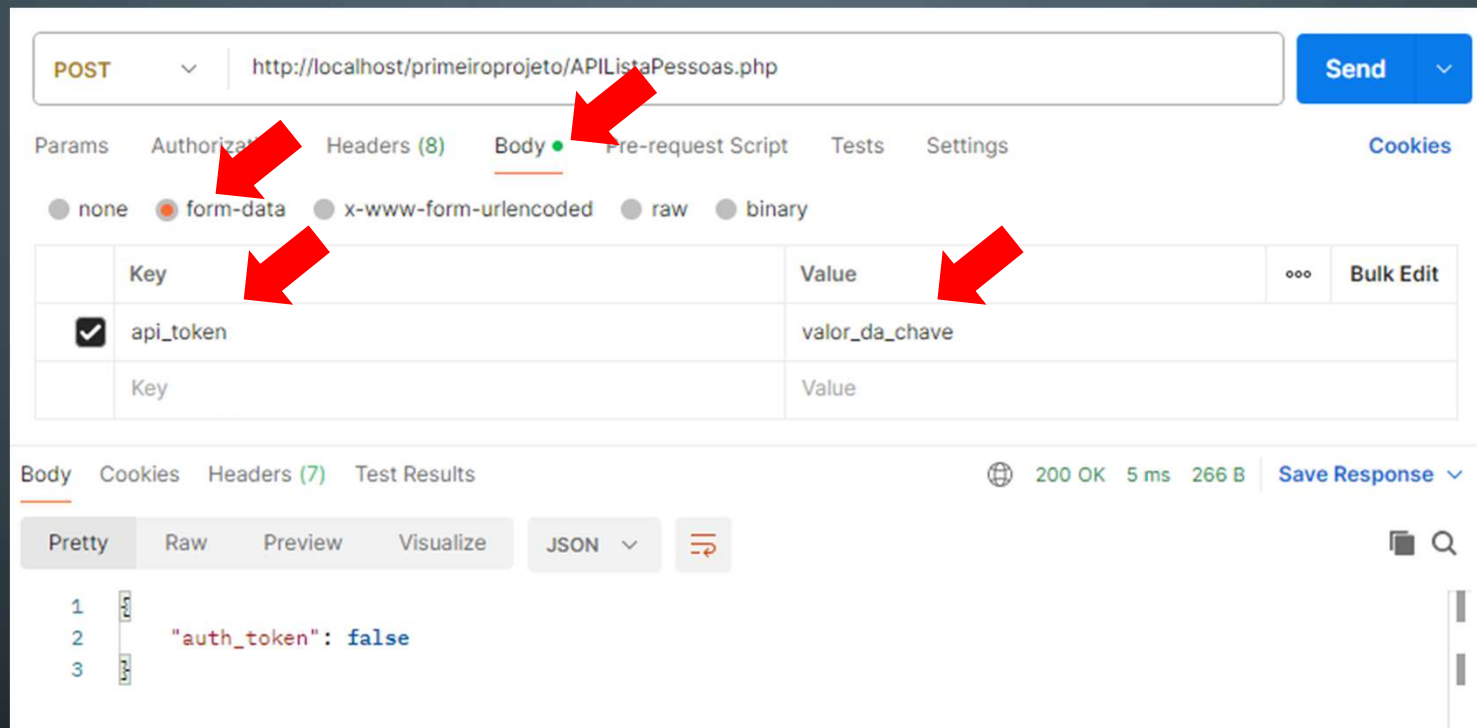
The screenshot shows a Postman interface for a POST request to `http://localhost/primeiroprojeto/APIListaPessoas.php`. The request is sent, and the response is displayed in the 'Body' tab. The response status is 200 OK, with a time of 12 ms and a size of 408 B. The response body is a JSON object:

```
1 <br />
2 <b>Warning</b>: Undefined array key "api_token" in <b>C:\xampp\htdocs\primeiroprojeto\APIListaPessoas.php</b> on line <b>7</b><br />
3 {
4   "auth_token": false
5 }
```

BACK END – TOKEN

- Este erro se deve ao fato de nossa API estar esperando receber uma variável (**api_token**) que não esta sendo enviada.
- Para enviá-la, clique em Body, em form-data e preencha os valores:
 - **Key:** nome da chave, no caso api_token
 - **Value:** valor que será enviado para essa chave.

BACK END – TOKEN



- Note que enviei um valor errado para a chave, o que retornou a mensagem de erro de autenticação.

BACK END – TOKEN

```
1  [
2    "auth_token": false
3  ]
```

- Por que retornamos uma variável com um valor dentro de um JSON e não uma simples mensagem de erro?
 - Porque na nossa aplicação esse valor será recebido e tratado pelo Front-End, e não será visto diretamente pelo usuário.
 - Estamos vendo a resposta pois estamos utilizando uma ferramenta (Postman) para testes do Back-End

BACK END – TOKEN

- Ao passar o valor correto ele traz os dados:

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** `http://localhost/primeiroprojeto/APIListaPessoas.php`
- Body Type:** form-data
- Body Parameters:**

Key	Value
☒ api_token	TokendeTeste
Key	Value
- Response:** 200 OK, 15 ms, 363 B
- Response Body (JSON):**

```
{  "id": 1,  "nome": "Huguinho",  "idade": 18}
```